User Defined Data Types

Introduction 2
Structure in C 2
Different Approaches of Creating Structures 4
Approach 1: Declare Structure Separately 4
Approach 2: Declare Variables During Structure Creation 4
Approach 3: Anonymous Structure 5
Structure Pointer 6
Passing Structure as Argument to Function 7
Copy One Structure to Another 8
Array of Structures 11
typedef in C 13
typedef with Structure 14
typedef with Pointer 14
Memory Layout of Structure 15
Structure Padding
Definition 15
Reducing Structure Padding 18
Method 1: Rearranging Members 18
Method 2: Using pragma pack 19
Method 3: Using attribute((packed)) 19
Disadvantages of Removing Padding 19
Structure Padding in Embedded Systems 19
Bit Field in Embedded Systems 21
Union in C 22
Union in Embedded Systems 24
Enum in C 25
Enum in Embedded Systems 26
Practice Questions 27
Structure Questions 27
typedef Questions 27
Union Questions 27
Enum Questions 28
Practice Programs 28
Tricky Interview MCQs 28

Introduction

In C programming, User Defined Data Types allow programmers to create their own data types according to application requirements.

These are heavily used in:

- Embedded Systems
- Device Driver Development
- Communication Protocols
- Register Mapping
- Real-Time Systems
- Sensor Data Handling
- Operating Systems

The major user-defined data types in C are:

1. Structure (struct)
2. Union (union)
3. Enumeration (enum)
4. typedef

Structure in C

Definition:

A structure is a user-defined data type used to group different data types of variables under a single name.

A structure can contain:

- int
- char
- float
- arrays
- pointers
- another structure

All members of a structure get separate memory locations.

Syntax of Structure

```
struct structure_name
{
   data_type member1;
   data_type member2;
   data_type member3;
};
```

Example of Structure

```c
#include<stdio.h>

// Define a structure named Student
struct Student
{
    int roll;          // Store student's roll number
    char name[20];     // Store student's name
    float marks;       // Store student's marks
};

int main()
{
    // Declare a structure variable s1
    struct Student s1;

    // Assign values to structure members
    s1.roll = 101;
    s1.marks = 89.5;

    // Display the roll number
    printf("Roll = %d\n", s1.roll);

    // Display the marks with 2 digits after decimal
    printf("Marks = %.2f\n", s1.marks);

    return 0;
}
```

Why Structures are Important in Embedded Systems

Structures are widely used in embedded systems for:

- Peripheral Register Mapping
- Sensor Data Organization
- Communication Frames
- CAN Protocol
- UART Packets
- GPIO Configuration
- RTC Data Storage

Example:

```
struct SensorData
{
   int temperature;
   int humidity;
   int pressure;
};
```

Different Approaches of Creating Structures

Approach 1: Declare Structure Separately

```
struct Employee
{
   int id;
   float salary;
};

struct Employee e1;
```

Approach 2: Declare Variables During Structure Creation

```
struct Employee
{
   int id;
   float salary;
}e1, e2;
```

Approach 3: Anonymous Structure

```
struct
{
    int x;
    int y;
}p1;
```

Accessing Structure Members

- Structure members are accessed using the dot (.) operator.
  Example : s1.roll = 10;
- A **pointer to a structure** accesses its members using the **arrow (->) operator**.

Printing Structure Members

```
#include<stdio.h>

struct Student
{
    int roll;
    float marks;
};

int main()
{
    struct Student s1;

    s1.roll = 1;
    s1.marks = 90.5;

    printf("Roll = %d\n", s1.roll);
    printf("Marks = %.2f", s1.marks);

    return 0;
}
```

Structure Pointer

```c
#include <stdio.h>

struct Student
{
    int roll;
    char name[20];
    float marks;
};

int main()
{
    // Declare and initialize structure variable
    struct Student s1 = {101, "Rahul", 89.5};

    // Create a pointer to the structure
    struct Student *ptr;

    // Store the address of structure variable in pointer
    ptr = &s1;

    // Access structure members using structure pointer
    printf("Roll = %d\n", ptr->roll);
    printf("Name = %s\n", ptr->name);
    printf("Marks = %.2f\n", ptr->marks);

    return 0;
}
```

Arrow Operator in Structure

The arrow operator (->) is used to access structure members using structure pointer.

ptr->roll is equivalent to a (*ptr).roll.

Passing Structure as Argument to Function

Method 1: Pass Entire Structure

```c
#include<stdio.h>

// Define a structure named Student
struct Student
{
    int roll;       // Store roll number
    float marks;    // Store marks
};

// Function to display student details
// Structure variable is passed as an argument
void display(struct Student s)
{
    printf("Roll = %d\n", s.roll);          // Display roll number
    printf("Marks = %.2f\n", s.marks);      // Display marks
}

int main()
{
    // Declare and initialize structure variable
    struct Student s1 = {1, 88.5};

    // Pass structure variable to the display function
    display(s1);

    return 0;
}
```

Method 2: Pass Structure Using Pointer

```c
#include <stdio.h>

struct Student
{
    int roll;
    char name[50];
```

```
    float marks;
};

// Function to display student details using a structure pointer
void display(struct Student *ptr)
{
    printf("\nStudent Details:\n");
    printf("Roll  = %d\n", ptr->roll);
    printf("Name  = %s", ptr->name);
    printf("Marks = %.2f\n", ptr->marks);
}

int main()
{
    struct Student s1;

    // Take student details from the user
    printf("Enter roll number: ");
    scanf("%d", &s1.roll);

    // Remove the newline left by scanf()
    getchar();

    printf("Enter name: ");
    fgets(s1.name, sizeof(s1.name), stdin);

    printf("Enter marks: ");
    scanf("%f", &s1.marks);

    // Pass the address of structure variable to the function
    display(&s1);

    return 0;
}
```

Copy One Structure to Another

```
#include <stdio.h>
```

```c
// Define a structure named Student
struct Student
{
    int roll;          // Store roll number
    char name[20];     // Store student name
    float marks;       // Store marks
};

int main()
{
    struct Student s1, s2;

    // Take student details from the user
    printf("Enter roll number: ");
    scanf("%d", &s1.roll);

    printf("Enter name: ");
    scanf("%19s", s1.name);

    printf("Enter marks: ");
    scanf("%f", &s1.marks);

    // Copy all members of s1 into s2
    s2 = s1;

    // Display the copied structure data
    printf("\nStudent 2 Details:\n");
    printf("Roll = %d\n", s2.roll);
    printf("Name = %s\n", s2.name);
    printf("Marks = %.2f\n", s2.marks);

    return 0;
}
```

Nested Structure

Definition

A structure inside another structure is called nested structure.

Example

```c
#include <stdio.h>

struct Date
{
    int day;
    int month;
    int year;
};

struct Student
{
    int roll;
    char name[50];
    struct Date dob;     // Nested structure
};

int main()
{
    struct Student s1;

    // Take student details from the user
    printf("Enter roll number: ");
    scanf("%d", &s1.roll);

    printf("Enter student name: ");
    scanf(" %[^\n]", s1.name);

    // Take date of birth from the user
    printf("Enter date of birth (DD MM YYYY): ");
    scanf("%d %d %d", &s1.dob.day, &s1.dob.month, &s1.dob.year);

    // Display student details
    printf("\nStudent Details:\n");
    printf("Roll = %d\n", s1.roll);
    printf("Name = %s\n", s1.name);
    printf("DOB  = %02d/%02d/%d\n",
           s1.dob.day,
           s1.dob.month,
           s1.dob.year);
```

```
    return 0;
}
```

Array of Structures

Example 1:

```
#include<stdio.h>

struct Student
{
   int roll;
   float marks;
};

int main()
{
   struct Student s[3];

   s[0].roll = 1;
   s[0].marks = 90;

   s[1].roll = 2;
   s[1].marks = 85;

   printf("%d %.2f", s[1].roll, s[1].marks);

   return 0;
}
```

Example 2:

```
#include <stdio.h>
#include <string.h>

struct Student
{
    int roll;
```

```
    char name[50];
    float marks;
};

int main()
{
    struct Student s[3];
    char tempName[50];
    int i;

    // Take details of 3 students from the user
    for (i = 0; i < 3; i++)
    {
        printf("\nEnter details of Student %d:\n", i + 1);

        printf("Enter roll number: ");
        scanf("%d", &s[i].roll);

        // Clear the newline left by scanf()
        getchar();

        printf("Enter name: ");
        fgets(tempName, sizeof(tempName), stdin);

        // Remove newline from the entered name
        tempName[strcspn(tempName, "\n")] = '\0';

        // Copy the name into the structure
        strcpy(s[i].name, tempName);

        printf("Enter marks: ");
        scanf("%f", &s[i].marks);
    }

    // Display details of all students
    printf("\nStudent Details:\n");

    for (i = 0; i < 3; i++)
    {
        printf("\nStudent %d:\n", i + 1);
        printf("Roll  = %d\n", s[i].roll);
        printf("Name  = %s\n", s[i].name);
```

```
        printf("Marks = %.2f\n", s[i].marks);
    }

    return 0;
}
```

typedef in C

Definition

typedef is used to create an alias name for existing data types.

It improves:

- readability
- portability
- maintainability

Syntax of typedef

typedef existing_data_type new_name;

Example of typedef

```
#include <stdio.h>

// Create an alias 'uint' for the data type 'unsigned int'
typedef unsigned int uint;

int main()
{
    // Declare a variable of type 'uint' and initialize it to 10
    uint x = 10;

    // Print the unsigned integer value
    printf("%u", x);

    return 0;
}
```

typedef with Structure

```c
#include <stdio.h>

// Define an anonymous structure and create an alias 'Student'
typedef struct
{
    int roll;       // Store roll number
    float marks;    // Store marks
} Student;

int main()
{
    // Declare a structure variable using the alias 'Student'
    Student s1;

    // Assign values to structure members
    s1.roll = 1;
    s1.marks = 88.5;

    // Display roll number and marks
    printf("%d %.2f", s1.roll, s1.marks);

    return 0;
}
```

typedef with Pointer

```c
#include <stdio.h>
// Create an alias 'ptr' for the data type 'int pointer'
typedef int* ptr;

int main()
{
    int x = 10;
```

```
    // Declare a pointer variable using the alias 'ptr'
    // and store the address of x
    ptr p = &x;

    // Dereference the pointer to access the value of x
    printf("%d", *p);

    return 0;
}
```

typedef in Embedded Systems

Widely used for:

- Register mapping
- Fixed width integers
- Driver development
- Portable code

Example:

typedef unsigned char uint8_t;
typedef unsigned int uint32_t;

Memory Layout of Structure

Structure members are stored sequentially in memory. However, the compiler may insert unused bytes called padding.

Structure Padding

Definition

Extra unused bytes added by the compiler between structure members for alignment purposes are called structure padding. Padding improves CPU access speed.

Example of Structure Padding

```c
#include<stdio.h>

struct Test
{
    char a;
    int b;
};

int main()
{
    printf("Size = %d", sizeof(struct Test));

    return 0;
}
```

Expected Calculation

Char = 1 byte
int   = 4 bytes
Total = 5 bytes

But actual output may be: 8 bytes

This is because of structure padding.

Why Padding Happens?

- **Data alignment** means storing a variable at a memory address that is suitable for its data type. Processors can access aligned data more efficiently because the data is placed at an address that matches the processor's natural memory access boundaries.
- For example, on a typical system, a `char` requires 1-byte alignment, a `short` requires 2-byte alignment, an `int` requires 4-byte alignment, and a `double` may require 8-byte alignment.
- This means a 4-byte `int` would preferably start at an address such as `0x1000`, `0x1004`, `0x1008`, or `0x100C`, because these addresses are multiples of 4. Similarly, a 2-byte `short` would preferably start at an even address such as `0x1000`, `0x1002`, or `0x1004`.
- If data is not properly aligned, some processors may need additional memory accesses to read the complete value, which can reduce performance. Some processors can handle unaligned data automatically, while others may generate an exception or require special handling.

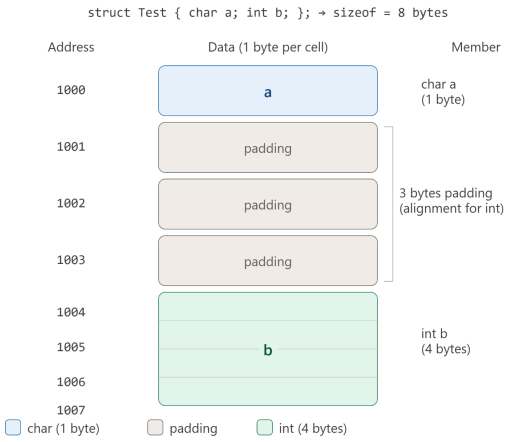
- Alignment is especially important in **structures**, where the compiler may insert unused bytes called **padding** between members to ensure that each member starts at a properly aligned address.
- For example, in `struct Test { char c; int i; };`, the `char` occupies 1 byte, but the compiler may insert 3 bytes of padding before the `int` so that the `int` starts at a 4-byte-aligned address.
- Therefore, **size tells us how much memory a data type occupies, while alignment tells us where that data should preferably start in memory**.

Example:

- int should start at multiple of 4
- short at multiple of 2
- Char at multiple of 1
- Double at multiple of 8

Compiler inserts padding bytes automatically.

Memory Layout Example

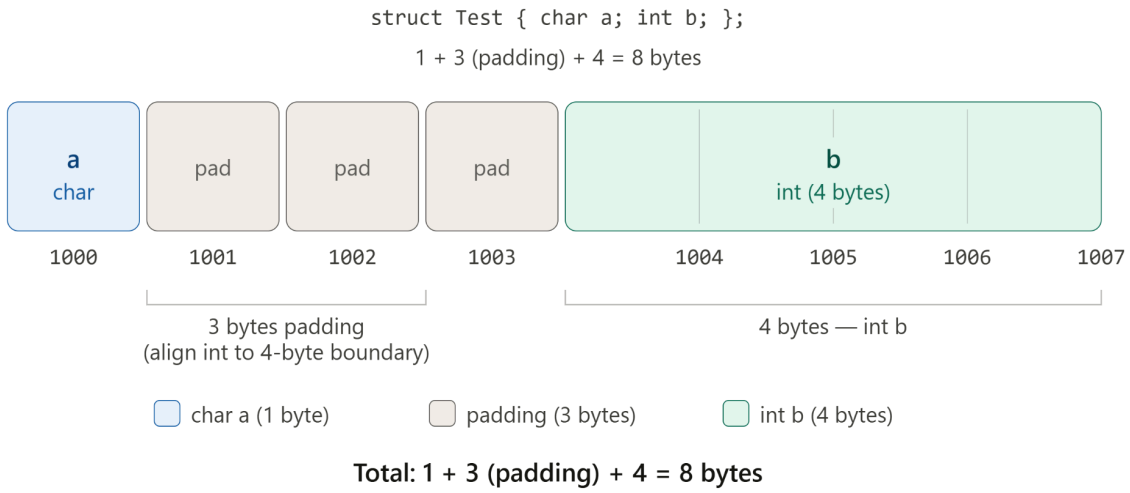


Total = 8 bytes

Another Padding Example

```
struct Demo
{
   char a;
   char b;
   int c;
};
```

Memory Layout:



Total: 1 + 3 (padding) + 4 = 8 bytes

Reducing Structure Padding

Method 1: Rearranging Members

```
struct Test
{
   int b;
   char a;
};
```

Now size may become: 8 bytes or 4+1+padding

Better optimization:

```
struct Test
{
   int a;
   int b;
   char c;
};
```

Method 2: Using pragma pack

```
#pragma pack(1)

struct Test
{
   char a;
   int b;
};
```

Now size becomes: 5 bytes

Method 3: Using attribute((packed))

```
struct __attribute__((packed)) Test
{
   char a;
   int b;
};
```

Disadvantages of Removing Padding

- Slower memory access
- More CPU cycles
- Misaligned access problems
- Some processors may generate fault

Structure Padding in Embedded Systems

Padding is very important in:

- UART Frames
- CAN Packets
- SPI Communication
- EEPROM Storage
- Network Protocols

Because exact byte arrangement matters.

Bit Fields in Structure

Definition

Bit fields allow allocation of specific number of bits to structure members.

Useful in:

- Hardware register access
- Protocol decoding
- Memory optimization

Syntax of Bit Field

```
struct structure_name
{
   data_type member : number_of_bits;
};
```

Example of Bit Field

```c
#include<stdio.h>

struct Status
{
   unsigned int ready : 1;
   unsigned int error : 1;
   unsigned int mode  : 2;
};

int main()
{
   struct Status s;

   s.ready = 1;
   s.error = 0;
   s.mode = 3;

   printf("Ready = %d\n", s.ready);
   printf("Error = %d\n", s.error);
   printf("Mode = %d\n", s.mode);

   return 0;
}
```

Bit Field in Embedded Systems

Used for:

- Status Registers
- Control Registers
- Peripheral Configuration
- Communication Protocols

Hardware Register Mapping Example

```
struct GPIO
{
   unsigned int pin0 : 1;
   unsigned int pin1 : 1;
   unsigned int pin2 : 1;
   unsigned int pin3 : 1;
};
```

Example of Bitfield

```c
#include <stdio.h>

struct Status
{
   unsigned int power : 1;    // 1 bit
   unsigned int mode  : 2;    // 2 bits
   unsigned int error : 1;    // 1 bit
};

int main()
{
   struct Status s;

   s.power = 1;
   s.mode  = 3;
   s.error = 0;

   printf("Power = %u\n", s.power);
   printf("Mode  = %u\n", s.mode);
```

```
    printf("Error = %u\n", s.error);

    return 0;
}
```

Why use bit-fields?

Instead of using a full unsigned int for each variable, bit-fields allow storing data using only the required number of bits, which helps save memory and is useful for hardware register representation in embedded systems.

Advantages of Bit-Fields

1. Saves memory.
2. Easy access to individual bits.
3. Useful for hardware register manipulation.
4. Improves code readability.

Limitations

1. Bit ordering is compiler-dependent.
2. Cannot take the address of a bit-field member.
3. Access may be slower on some processors due to extra masking operations.

Union in C

Definition

Union is a user-defined data type where all members share the same memory location.

Only one member can hold valid data at a time.

Syntax of Union

```
union union_name
{
    data_type member1;
    data_type member2;
};
```

Example of Union

```c
#include<stdio.h>

union Data
{
    int x;
    char ch;
};

int main()
{
    union Data d;

    d.x = 65;

    printf("x = %d\n", d.x);

    d.ch = 'A';

    printf("ch = %c\n", d.ch);

    return 0;
}
```

Size of Union

Size of union equals the size of largest member.

Example:

```
union Test
{
    int a;
    char b;
    float c;
};
```

Largest member: float = 4 bytes

Union size: 4 bytes

Structure vs Union

Feature	Structure	Union
Memory	Separate	Shared
Size	Sum of members	Largest member
Access	All members simultaneously	One member at a time
Use	Grouping data	Memory optimization

Union in Embedded Systems

Used in:

- Memory constrained systems
- Protocol decoding
- Register access
- Data interpretation

Register Mapping Using Union

```
#include<stdio.h>

union Register
{
    unsigned int reg;

    struct
    {
        unsigned int bit0 : 1;
        unsigned int bit1 : 1;
        unsigned int bit2 : 1;
    }bits;
};
```

Enum in C

Definition :

Enumeration is a user-defined data type consisting of named integer constants.

Syntax of Enum

```
enum enum_name
{
   constant1,
   constant2,
   constant3
};
```

Example of Enum

```c
#include<stdio.h>

enum Day
{
   MON,
   TUE,
   WED,
   THU,
   FRI
};

int main()
{
   enum Day d;

   d = WED;

   printf("%d", d);

   return 0;
}
```

Output: 2

Custom Enum Values

```c
#include<stdio.h>

enum Status
{
    OFF = 0,
    ON = 1,
    ERROR = -1
};
```

Enum in Embedded Systems

Used for:

- State Machines
- Error Codes
- Peripheral Modes
- Device States
- Protocol Commands

Example: LED State Machine

```c
#include<stdio.h>

enum LED_State
{
   LED_OFF,
   LED_ON,
   LED_BLINK
};

int main()
{
   enum LED_State led;

   led = LED_BLINK;

   if(led == LED_BLINK)
```

```
    {
        printf("LED Blinking");
    }

    return 0;
}
```

Practice Questions

Structure Questions

1. What is structure in C?
2. Difference between structure and union?
3. What is structure padding?
4. Why compiler adds padding?
5. How to remove structure padding?
6. What is nested structure?
7. Difference between structure pointer and normal pointer?
8. Explain arrow operator.
9. What is bit field?
10. Can structures be copied directly?

typedef Questions

1. Why typedef is used?
2. typedef vs #define
3. typedef with pointers.
4. typedef with structures.
5. Why typedef is heavily used in embedded systems?

Union Questions

1. Why union size equals largest member?
2. When to use union?
3. Difference between union and structure?
4. How unions save memory?
5. Can multiple members hold values simultaneously?

Enum Questions

1. What is enum?
2. Default enum values?
3. Can enum values be modified?
4. Why enum used in embedded systems?
5. enum vs macro.

Practice Programs

1. Create structure for employee details.
2. Pass structure to function.
3. Create nested structure.
4. Create array of structures.
5. Calculate structure size.
6. Remove structure padding.
7. Create bit field for hardware register.
8. Implement LED state machine using enum.
9. Create union for sensor data.
10. Use typedef with structure.

Tricky Interview MCQs

Q1.

```
struct Test
{
   char a;
   int b;
};
```

Possible size?

A) 5
B) 6
C) 8
D) 4

Answer: C

Q2.

Which operator is used with structure pointer?

A) .
B) ::
C) ->
D) *

Answer: C

Q3.

In union:

A) All members have separate memory
B) Members share memory
C) No memory allocated
D) Dynamic memory allocated

Answer: B

Q4.

Default first enum value is:

A) 1
B) -1
C) 0
D) Garbage

Answer: C

Q5.

Which is best for register mapping?

A) Array
B) Structure with bit fields
C) Pointer
D) Function

Answer: B